



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

## ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

### Лабораторная работа № 9

на тему:

«Игры на разорение. Нахождение цены игры и оптимальных  
смешанных стратегий»

Вариант 13

Студент:

ИУ8-105  
(Группа)

Б. Е. Шекунов  
(И. О. Фамилия)

Преподаватель:

Н. С. Коннова  
(И. О. Фамилия)

Москва, 2026 г.

### **Цель работы:**

Изучить игры на разорение и рекурсивные матричные игры. Решить индивидуальный вариант рекурсивной игры, найдя цену игры и оптимальные смешанные стратегии игроков. Дополнительно рассмотреть игру «женщины и кошки против мышей и мужчин» для проверки работы метода на другой постановке игры на разорение.

### **Постановка задачи:**

В игре на разорение требуется последовательно вычислять цену игры  $v_N$ , где на каждом шаге строится новая матрица игры  $\Gamma_N$ , зависящая от результата предыдущего шага. Для игры на разорение требуется вычислить цену игры и оптимальные смешанные стратегии игроков.

Используется матрица игры  $\Gamma_N$ :

$$\Gamma_N = \begin{pmatrix} -5 & 7 \\ 11 & \Gamma_{N-1} \end{pmatrix}.$$

На каждом шаге вместо  $\Gamma_{N-1}$  подставляется ранее найденная цена игры  $v_{N-1}$ . Тогда матрица очередного шага имеет вид:

$$\Gamma_N = \begin{pmatrix} -5 & 7 \\ 11 & v_{N-1} \end{pmatrix}.$$

Начальное значение  $v_1 = 0$ .

Требуется вычислить цену игры  $v_{13}$ , определить нижнюю и верхнюю цены игры на последнем шаге, проверить наличие седловой точки и найти оптимальные смешанные стратегии игроков.

Для второй задачи рассматривается игра между двумя группами:

$$v(m_1, m_2; n_1, n_2),$$

где:

$m_1$  – число женщин,

$m_2$  – число кошек,

$n_1$  – число мышей,

$n_2$  – число мужчин.

Цена игры находится рекурсивно через значения подыгр.

**Исходные данные:**

Рекурсивная игра:

$$\Gamma_N = \begin{pmatrix} -5 & 7 \\ 11 & \Gamma_{N-1} \end{pmatrix}.$$

Начальное значение  $v_1 = 0$ .

Расчёт выполняется до  $N = 13$ .

Вариант для игры «женщины и кошки против мышей и мужчин»

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3.$$

То есть необходимо найти:

$$v(3,4; 2,3).$$

**Ход работы:**

Используется рекурсивная игра:

$$\Gamma_N = \begin{pmatrix} -5 & 7 \\ 11 & v_{N-1} \end{pmatrix}.$$

Начальное значение  $v_1 = 0$ .

Пусть матрица игры имеет вид:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

Если седловая точка отсутствует, цена игры в смешанных стратегиях вычисляется по формуле:

$$v = \frac{ad - bc}{a - b - c + d}.$$

В нашем случае:

$$a = -5, b = 7, c = 11, d = v_{N-1}.$$

Тогда:

$$v_N = \frac{(-5)v_{N-1} - 7 \cdot 11}{-5 - 7 - 11 + v_{N-1}}.$$

$$v_N = \frac{-5v_{N-1} - 77}{v_{N-1} - 23}.$$

Умножим числитель и знаменатель на  $-1$ :

$$v_N = \frac{5v_{N-1} + 77}{23 - v_{N-1}}.$$

Таким образом, для индивидуального варианта используется рекурсия:

$$v_N = \frac{5v_{N-1} + 77}{23 - v_{N-1}}, v_1 = 0.$$

Последовательный расчёт значений  $v_N$

Для  $N = 2$ :

$$v_2 = \frac{5v_1 + 77}{23 - v_1} = \frac{5 \cdot 0 + 77}{23 - 0} = \frac{77}{23} \approx 3.347826.$$

Для  $N = 3$ :

$$v_3 = \frac{5v_2 + 77}{23 - v_2} = \frac{5 \cdot \frac{77}{23} + 77}{23 - \frac{77}{23}} = \frac{539}{113} \approx 4.769912.$$

Для  $N = 4$ :

$$v_4 = \frac{2849}{515} \approx 5.532039.$$

Для  $N = 5$ :

$$v_5 = \frac{13475}{2249} \approx 5.991552.$$

Для  $N = 6$ :

$$v_6 = \frac{60137}{9563} \approx 6.288508.$$

Для  $N = 7$ :

$$v_7 = \frac{259259}{39953} \approx 6.489100.$$

Для  $N = 8$ :

$$v_8 = \frac{1093169}{164915} \approx 6.628681.$$

Для  $N = 9$ :

$$v_9 = \frac{4541075}{674969} \approx 6.727828.$$

Для  $N = 10$ :

$$v_{10} = \frac{18669497}{2745803} \approx 6.799285.$$

Для  $N = 11$ :

$$v_{11} = \frac{76193579}{11120993} \approx 6.851329.$$

Для  $N = 12$ :

$$v_{12} = \frac{309321089}{44897315} \approx 6.889523.$$

Для  $N = 13$ :

$$v_{13} = \frac{1250924675}{180829289} \approx 6.917711.$$

На последнем шаге используется значение:

$$v_{12} = \frac{309321089}{44897315} \approx 6.889523.$$

Поэтому матрица игры имеет вид:

$$\Gamma_{13} = \begin{pmatrix} -5 & 7 \\ 11 & 6.89 \end{pmatrix}.$$

Нижняя цена игры:

$$\underline{v} = \frac{309321089}{44897315} \approx 6.889523.$$

Верхняя цена игры:

$$\bar{v} = 7.$$

Так как:

$$\underline{v} < \bar{v},$$

седловой точки в чистых стратегиях нет. Следовательно, решение ищется в смешанных стратегиях.

Цена игры на последнем шаге:

$$v_{13} = \frac{1250924675}{180829289} \approx 6.917711.$$

Оптимальная стратегия игрока-строки:

$$x = \left( \frac{46137344}{180829289}, \frac{134691945}{180829289} \right).$$

В десятичном виде:

$$x \approx (0.255143, 0.744857).$$

Оптимальная стратегия игрока-столбца:

$$y = \left( \frac{1240029}{180829289}, \frac{179589260}{180829289} \right).$$

В десятичном виде:

$$y \approx (0.006857, 0.993143).$$

Вариант «женщины и кошки против мышей и мужчин»

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3.$$

Необходимо найти:

$$v(3,4; 2,3).$$

Для построения матрицы очередного хода были вычислены значения подыгр:

$$v(m_2 - 1) = v(3,3; 2,3) = \frac{2}{7} \approx 0.285714,$$

$$v(n_1 - 1) = v(3,4; 1,3) = \frac{16}{21} \approx 0.761905,$$

$$v(n_2 - 1) = v(3,4; 2,2) = \frac{2}{3} \approx 0.666667,$$

$$v(m_1 - 1) = v(2,4; 2,3) = \frac{4}{21} \approx 0.190476.$$

Матрица очередного хода:

|              | <i>man</i>      | <i>mouse</i>  |
|--------------|-----------------|---------------|
| <i>cat</i>   | $\frac{4}{21}$  | $\frac{2}{3}$ |
| <i>woman</i> | $\frac{16}{21}$ | $\frac{2}{7}$ |

Нижняя цена игры:

$$\underline{v} = \frac{2}{7} \approx 0.285714.$$

Верхняя цена игры:

$$\overline{v} = \frac{2}{3} \approx 0.666667.$$

Седловой точки нет, поэтому игра решается в смешанных стратегиях.

Цена игры:

$$v(3,4; 2,3) = \frac{10}{21} \approx 0.47619.$$

Оптимальная стратегия группы I:

$$x = (cat, woman) = \left(\frac{1}{2}, \frac{1}{2}\right).$$

Оптимальная стратегия группы II:

$$y = (man, mouse) = \left(\frac{2}{5}, \frac{3}{5}\right).$$

Проверим значение по рекуррентной формуле:

$$v = \frac{v(m_1 - 1)v(m_2 - 1) - v(n_1 - 1)v(n_2 - 1)}{v(m_1 - 1) + v(m_2 - 1) - v(n_1 - 1) - v(n_2 - 1)}.$$

Подстановка вычисленных значений даёт:

$$v = \frac{10}{21}.$$

Значение совпадает с ценой матричной игры.

### Вывод:

В работе были рассмотрены игры на разорение и рекурсивные матричные игры.

Для заданной по варианту матрицы игры  $\Gamma_N$  получена цена игры:

$$v_{13} = \frac{1250924675}{180829289} \approx 6.917711.$$

На последнем шаге нижняя и верхняя цены игры не совпадают:

$$\underline{v} \approx 6.889523, \bar{v} = 7.$$

Следовательно, седловая точка в чистых стратегиях отсутствует, и решение найдено в смешанных стратегиях.

Оптимальная стратегия игрока-строки:

$$x = \left( \frac{46137344}{180829289}, \frac{134691945}{180829289} \right) \approx (0.255143, 0.744857).$$

Оптимальная стратегия игрока-столбца:

$$y = \left( \frac{1240029}{180829289}, \frac{179589260}{180829289} \right) \approx (0.006857, 0.993143).$$

Для игры «женщины и кошки против мышей и мужчин» при параметрах:

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3$$

получена цена игры:

$$v(3,4; 2,3) = \frac{10}{21} \approx 0.47619.$$

Оптимальная стратегия группы I:

$$(cat, woman) = \left( \frac{1}{2}, \frac{1}{2} \right).$$

Оптимальная стратегия группы II:

$$(man, mouse) = \left( \frac{2}{5}, \frac{3}{5} \right).$$

Таким образом, индивидуальный вариант рекурсивной игры решён корректно: на каждом шаге матрица  $2 \times 2$  была сведена к матричной игре с нулевой суммой, а на последнем шаге получены цена игры и оптимальные смешанные стратегии игроков.

## Приложение (исходный код)

```
from __future__ import annotations
from dataclasses import dataclass
from functools import lru_cache
from fractions import Fraction
from typing import Hashable, Sequence

Number = int | float | Fraction
PREV = "PREV"

def F(x: Number) -> Fraction:
    if isinstance(x, Fraction):
        return x
    return Fraction(x).limit_denominator()

def fmt_frac(x: Number, show_decimal: bool = True, digits: int = 6) -> str:
    value = F(x)
    if value.denominator == 1:
        base = str(value.numerator)
    else:
        base = f"{value.numerator}/{value.denominator}"

    if not show_decimal:
        return base

    dec = float(value)
    if abs(dec - round(dec)) < 10 ** (-(digits - 1)):
        return base
    return f"{base} ≈ {dec:.{digits}f}".rstrip("0").rstrip(".")

def fmt_strategy(strategy: tuple[Fraction, Fraction], labels: tuple[str, str]) -> str:
    return (
        f"({labels[0]}: {fmt_frac(strategy[0])}); "
        f"{labels[1]}: {fmt_frac(strategy[1])})"
    )

def print_matrix_2x2(
    matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]],
    row_labels: tuple[str, str] = ("1", "2"),
    col_labels: tuple[str, str] = ("1", "2"),
) -> None:
    width = 20
    print(f"{'':>14}{col_labels[0]:>{width}}{col_labels[1]:>{width}}")
    for label, row in zip(row_labels, matrix):
        print(f"{label:>14}{fmt_frac(row[0]):>{width}}{fmt_frac(row[1]):>{width}}")

@dataclass(frozen=True)
class Matrix2x2Solution:
    matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]
    value: Fraction
    row_strategy: tuple[Fraction, Fraction]
    col_strategy: tuple[Fraction, Fraction]
    lower_price: Fraction
    upper_price: Fraction
    saddle_points: tuple[tuple[int, int], ...]
    method: str

def solve_zero_sum_2x2(matrix_like: Sequence[Sequence[Number]]) -> Matrix2x2Solution:
    if len(matrix_like) != 2 or any(len(row) != 2 for row in matrix_like):
        raise ValueError("Ожидается матрица размера 2x2")

    matrix = tuple(tuple(F(x) for x in row) for row in matrix_like) # type: ignore[assignment]
    a, b = matrix[0]
    c, d = matrix[1]

    row_mins = (min(matrix[0]), min(matrix[1]))
    lower_price = max(row_mins)
    col_maxs = (max(matrix[0][0], matrix[1][0]), max(matrix[0][1], matrix[1][1]))
    upper_price = min(col_maxs)

    saddle_points: list[tuple[int, int]] = []
    if lower_price == upper_price:
        for i in range(2):
            for j in range(2):
                if matrix[i][j] == lower_price and matrix[i][j] == row_mins[i] and matrix[i][j]
    == col_maxs[j]:
```

```

        saddle_points.append((i, j))

    if saddle_points:
        i, j = saddle_points[0]
        row_strategy = (Fraction(1), Fraction(0)) if i == 0 else (Fraction(0), Fraction(1))
        col_strategy = (Fraction(1), Fraction(0)) if j == 0 else (Fraction(0), Fraction(1))
        return Matrix2x2Solution(
            matrix=matrix,
            value=lower_price,
            row_strategy=row_strategy,
            col_strategy=col_strategy,
            lower_price=lower_price,
            upper_price=upper_price,
            saddle_points=tuple(saddle_points),
            method="седловая точка, чистые стратегии",
        )

    denominator = a - b - c + d
    if denominator == 0:
        raise ValueError(
            "Внутреннее смешанное решение не найдено: знаменатель равен 0, "
            "а седловая точка отсутствует. Нужен отдельный анализ вырожденной игры."
        )

    p_row1 = (d - c) / denominator
    q_col1 = (d - b) / denominator
    value = (a * d - b * c) / denominator

    row_strategy = (p_row1, 1 - p_row1)
    col_strategy = (q_col1, 1 - q_col1)

    if any(x < 0 or x > 1 for x in row_strategy + col_strategy):
        raise ValueError(
            "Вероятности смешанного решения вышли за пределы [0,1]. "
            "Для этой матрицы требуется дополнительный анализ доминирования/вырождения."
        )

    return Matrix2x2Solution(
        matrix=matrix,
        value=value,
        row_strategy=row_strategy,
        col_strategy=col_strategy,
        lower_price=lower_price,
        upper_price=upper_price,
        saddle_points=tuple(),
        method="смешанные стратегии",
    )

def print_matrix_solution(
    title: str,
    matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]],
    row_labels: tuple[str, str] = ("строка 1", "строка 2"),
    col_labels: tuple[str, str] = ("столбец 1", "столбец 2"),
) -> Matrix2x2Solution:
    print("\n" + title)
    print_matrix_2x2(matrix, row_labels=row_labels, col_labels=col_labels)
    solution = solve_zero_sum_2x2(matrix)
    print(f"Метод решения: {solution.method}")
    print(f"Нижняя цена игры: {fmt_frac(solution.lower_price)}")
    print(f"Верхняя цена игры: {fmt_frac(solution.upper_price)}")
    print(f"Цена игры: {fmt_frac(solution.value)}")
    print(f"Оптимальная стратегия игрока-строки: {fmt_strategy(solution.row_strategy, row_labels)}")
    print(f"Оптимальная стратегия игрока-столбца: {fmt_strategy(solution.col_strategy, col_labels)}")
    if solution.saddle_points:
        points = ", ".join(f"({i + 1}, {j + 1})" for i, j in solution.saddle_points)
        print(f"Седловые точки: {points}")
    return solution

def inspection_value_recursive(N: int) -> Fraction:
    if N < 1:
        raise ValueError("N должно быть не меньше 1")
    value = Fraction(0)
    for _ in range(2, N + 1):
        value = (value + 1) / (-value + 3)
    return value

def inspection_value_closed(N: int) -> Fraction:

```

```

    if N < 1:
        raise ValueError("N должно быть не меньше 1")
    return Fraction(N - 1, N + 1)

def inspection_matrix(N: int) -> tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
    if N < 2:
        raise ValueError("Матрица  $\Gamma_N$  в виде 2x2 строится для  $N \geq 2$ ")
    previous = inspection_value_recursive(N - 1)
    return ((Fraction(-1), Fraction(1)), (Fraction(1), previous))

def inspection_optimal_strategy(N: int) -> tuple[Fraction, Fraction]:
    if N < 2:
        raise ValueError("Оптимальная стратегия в указанном виде дана для  $N \geq 2$ ")
    return Fraction(1, N + 1), Fraction(N, N + 1)

def print_inspection_lecture_check(max_N: int = 8) -> None:
    print("\n" + "=" * 100)
    print("Проверка лекционного примера: инспектирование")
    print("=" * 100)
    print("Рекурсия:  $v_1 = 0, \quad v_N = (v_{N-1} + 1) / (-v_{N-1} + 3)$ ")
    print("Замкнутая формула:  $v_N = (N-1) / (N+1)$ ")
    print("Оптимальные стратегии при  $N \geq 2$ :  $x^N = y^N = (1 / (N+1), N / (N+1))$ ")

    print("\nПроверочная таблица:")
    print(f"{'N':>3} {'v_N рекурсивно':>22} {'v_N формула':>22} {'цена матрицы  $\Gamma_N$ ':>22} {'x=y':>32}")
    for N in range(1, max_N + 1):
        v_rec = inspection_value_recursive(N)
        v_closed = inspection_value_closed(N)
        if N == 1:
            matrix_value = "-"
            strategy = "-"
        else:
            solution = solve_zero_sum_2x2(inspection_matrix(N))
            matrix_value = fmt_frac(solution.value)
            strategy = f"({fmt_frac(solution.row_strategy[0], False)}, {fmt_frac(solution.row_strategy[1], False)})"
        print(
            f"{'N':>3} {fmt_frac(v_rec):>22} {fmt_frac(v_closed):>22} "
            f"{matrix_value:>22} {strategy:>32}"
        )

RUIN_RECURSIVE_VARIANTS: dict[int, tuple[tuple[int | str, int | str], tuple[int | str, int | str]]] = {
    1: ((-5, 7), (8, PREV)),
    2: ((PREV, 5), (9, -3)),
    3: ((3, PREV), (-2, 1)),
    4: ((6, -3), (PREV, 4)),
    5: ((-5, 7), (11, PREV)),
    6: ((PREV, 9), (4, -1)),
    7: ((2, PREV), (-1, 3)),
    8: ((5, -4), (PREV, 8)),
}

def recursive_variant_number(student_variant: int) -> int:
    if student_variant < 1:
        raise ValueError("Номер варианта должен быть положительным")
    return ((student_variant - 1) % 8) + 1

def build_recursive_variant_matrix(
    variant_number: int,
    previous_value: Fraction,
) -> tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
    if variant_number not in RUIN_RECURSIVE_VARIANTS:
        raise ValueError("variant_number должен быть от 1 до 8")

    template = RUIN_RECURSIVE_VARIANTS[variant_number]
    matrix: list[tuple[Fraction, Fraction]] = []
    for row in template:
        new_row: list[Fraction] = []
        for item in row:
            if item == PREV:
                new_row.append(previous_value)
            else:
                new_row.append(F(item))
        matrix.append(tuple(new_row)) # type: ignore[arg-type]

```

```

        return tuple(matrix) # type: ignore[return-value]

def recursive_variant_value(
    variant_number: int,
    N: int,
    initial_value: Fraction = Fraction(0),
) -> Fraction:
    if N < 1:
        raise ValueError("N должно быть не меньше 1")

    value = initial_value
    for _ in range(2, N + 1):
        matrix = build_recursive_variant_matrix(variant_number, value)
        solution = solve_zero_sum_2x2(matrix)
        value = solution.value
    return value

def print_recursive_ruin_variant(
    student_variant: int,
    N: int,
    initial_value: Fraction = Fraction(0),
) -> None:
    variant_number = recursive_variant_number(student_variant)

    print("\n" + "=" * 100)
    print("Индивидуальный вариант с последнего слайда")
    print("=" * 100)
    print(f"Номер студенческого варианта: {student_variant}")
    print(f"Номер варианта на слайде: (({student_variant}-1) mod 8)+1 = {variant_number}")
    print(f"Расчет выполняется до N = {N}")
    print(f"Начальное значение: v 1 = {fmt_frac(initial_value)}")
    print(f"Шаблон матрицы варианта {variant_number}: {RUIN_RECURSIVE_VARIANTS[variant_number]}")

    value = initial_value
    print(f"\nv_1 = {fmt_frac(value)}")

    final_solution: Matrix2x2Solution | None = None

    for k in range(2, N + 1):
        matrix = build_recursive_variant_matrix(variant_number, value)

        print("\n" + "-" * 100)
        print(f"Шаг N = {k}")
        print(f"Подставляем v_{k-1} = {fmt_frac(value)} в матрицу Г_{k}:")
        print_matrix_2x2(matrix, row_labels=("I", "II"), col_labels=("I", "II"))

        solution = solve_zero_sum_2x2(matrix)
        final_solution = solution

        print(f"Метод решения: {solution.method}")
        print(f"Нижняя цена игры: {fmt_frac(solution.lower_price)}")
        print(f"Верхняя цена игры: {fmt_frac(solution.upper_price)}")
        print(f"Цена игры: v_{k} = {fmt_frac(solution.value)}")
        print(f"Оптимальная стратегия игрока-строки: {fmt_strategy(solution.row_strategy, ('I', 'II'))}")
        print(f"Оптимальная стратегия игрока-столбца: {fmt_strategy(solution.col_strategy, ('I', 'II'))}")

        value = solution.value

    print("\n" + "-" * 100)
    print("Итоговые значения для отчета:")
    print(f"Номер варианта на последнем слайде: {variant_number}")
    print(f"N = {N}")
    print(f"v_N = {fmt_frac(value)}")
    if final_solution is not None:
        print(
            f"Оптимальная стратегия игрока-строки на последнем шаге: {fmt_strategy(final_solution.row_strategy, ('I', 'II'))}"
        )
        print(
            f"Оптимальная стратегия игрока-столбца на последнем шаге: {fmt_strategy(final_solution.col_strategy, ('I', 'II'))}"
        )

@dataclass(frozen=True)
class WomenCatsMiceMenGame:
    @staticmethod
    def _validate_state(m1: int, m2: int, n1: int, n2: int) -> None:
        if min(m1, m2, n1, n2) < 0:

```

```

        raise ValueError("Количество участников не может быть отрицательным")

@lru_cache(maxsize=None)
def value(self, m1: int, m2: int, n1: int, n2: int) -> Fraction:
    self._validate_state(m1, m2, n1, n2)

    if (n1 == 0 or n2 == 0) and m1 > 0 and m2 > 0:
        return Fraction(1)
    if (m1 == 0 or m2 == 0) and n1 > 0 and n2 > 0:
        return Fraction(-1)
    if m1 == 0 or m2 == 0 or n1 == 0 or n2 == 0:
        return Fraction(0)

    matrix = self.subgame_matrix(m1, m2, n1, n2)
    return solve_zero_sum_2x2(matrix).value

def subgame_matrix(self, m1: int, m2: int, n1: int, n2: int) -> tuple[
    tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
    self._validate_state(m1, m2, n1, n2)
    if min(m1, m2, n1, n2) <= 0:
        raise ValueError("Матрица очередного хода строится только для положительных параметров")

    return (
        (
            self.value(m1, m2 - 1, n1, n2),
            self.value(m1, m2, n1 - 1, n2),
        ),
        (
            self.value(m1, m2, n1, n2 - 1),
            self.value(m1 - 1, m2, n1, n2),
        ),
    )

def solve_state(self, m1: int, m2: int, n1: int, n2: int) -> Matrix2x2Solution:
    return solve_zero_sum_2x2(self.subgame_matrix(m1, m2, n1, n2))

def recurrence_formula_value(self, m1: int, m2: int, n1: int, n2: int) -> Fraction:
    a = self.value(m1, m2 - 1, n1, n2)
    b = self.value(m1, m2, n1 - 1, n2)
    c = self.value(m1, m2, n1, n2 - 1)
    d = self.value(m1 - 1, m2, n1, n2)
    denominator = a + d - b - c
    if denominator == 0:
        raise ZeroDivisionError("Знаменатель рекуррентной формулы равен 0")
    return (a * d - b * c) / denominator

def generated_population_variant(variant: int) -> tuple[int, int, int, int]:
    m1 = 2 + (variant % 3)
    m2 = 2 + ((variant + 1) % 3)
    n1 = 2 + ((variant + 2) % 3)
    n2 = 2 + ((variant + 3) % 3)
    return m1, m2, n1, n2

def print_women_cats_lecture_check() -> None:
    print("\n" + "=" * 100)
    print("Дополнительная проверка: женщины и кошки против мышей и мужчин")
    print("=" * 100)
    print("Проверяется значение v(1,1;1,1), которое по лекции равно 0.")

    game = WomenCatsMiceMenGame()
    params = (1, 1, 1, 1)
    matrix = game.subgame_matrix(*params)
    solution = print_matrix_solution(
        title="Матрица для состояния v(1,1;1,1)",
        matrix=matrix,
        row_labels=("cat", "woman"),
        col_labels=("man", "mouse"),
    )
    print(f"Итог: v(1,1;1,1) = {fmt_frac(solution.value)}")

def print_women_cats_individual_variant(m1: int, m2: int, n1: int, n2: int) -> None:
    print("\n" + "=" * 100)
    print(f"Дополнительный расчет: v({m1},{m2};{n1},{n2})")
    print("=" * 100)

    game = WomenCatsMiceMenGame()
    matrix = game.subgame_matrix(m1, m2, n1, n2)

```

```

print("Смысл параметров:")
print(f"  Группа I: m1={m1} женщин, m2={m2} кошек")
print(f"  Группа II: n1={n1} мышей, n2={n2} мужчин")

print("\nЗначения подыгр, входящих в рекуррентную матрицу:")
print(f"  v(m2-1) = v({m1},{m2-1},{n1},{n2}) = {fmt_frac(matrix[0][0])}")
print(f"  v(n1-1) = v({m1},{m2},{n1-1},{n2}) = {fmt_frac(matrix[0][1])}")
print(f"  v(n2-1) = v({m1},{m2},{n1},{n2-1}) = {fmt_frac(matrix[1][0])}")
print(f"  v(m1-1) = v({m1-1},{m2},{n1},{n2}) = {fmt_frac(matrix[1][1])}")

solution = print_matrix_solution(
    title="Матрица очередного хода",
    matrix=matrix,
    row_labels=("cat", "woman"),
    col_labels=("man", "mouse"),
)

formula_value = game.recurrence_formula_value(m1, m2, n1, n2)
print("\nПроверка по рекуррентной формуле:")
print(f"  v = (v(m1-1)*v(m2-1) - v(n1-1)*v(n2-1)) /")
print(f"        (v(m1-1)+v(m2-1)-v(n1-1)-v(n2-1))")
print(f"  Значение по формуле: {fmt_frac(formula_value)}")
print(f"  Совпадение с ценой матричной игры: {'да' if formula_value == solution.value else 'нет'}")

print("\nИтоговые значения:")
print(f"  Цена игры: v({m1},{m2},{n1},{n2}) = {fmt_frac(solution.value)}")
print(f"  Оптимальная стратегия группы I: {fmt_strategy(solution.row_strategy, ('cat', 'woman'))}")
print(f"  Оптимальная стратегия группы II: {fmt_strategy(solution.col_strategy, ('man', 'mouse'))}")
print(f"  Количество вычисленных рекурсивных состояний: {game.value.cache_info().currsz}")

VARIANT = 13

SLIDE_VARIANT = recursive_variant_number(VARIANT)
RECURSIVE_N = VARIANT
INITIAL_VALUE = Fraction(0)

POPULATION_PARAMS = generated_population_variant(VARIANT)

def solve_ruin_games() -> None:
    print("Лабораторная работа: Игры на разорение")
    print(f"Студенческий вариант: {VARIANT}")
    print(f"Вариант с последнего слайда: {SLIDE_VARIANT}")
    print(f"N для рекурсивной игры: {RECURSIVE_N}")
    print(
        "Дополнительные параметры для игры 'женщины и кошки против мышей и мужчин': "
        f"m1={POPULATION_PARAMS[0]}, m2={POPULATION_PARAMS[1]}, "
        f"n1={POPULATION_PARAMS[2]}, n2={POPULATION_PARAMS[3]}"
    )

    print_inspection_lecture_check(max_N=8)

    print_recursive_ruin_variant(
        student_variant=VARIANT,
        N=RECURSIVE_N,
        initial_value=INITIAL_VALUE,
    )

    print_women_cats_lecture_check()
    print_women_cats_individual_variant(*POPULATION_PARAMS)

if __name__ == "__main__":
    solve_ruin_games()

```